

统正式替换现有系统。在并行工作期间，手工处理和计算机处理系统并存，一旦新系统有问题就可以暂时停止而不会影响现有系统的正常工作。并行转换过程如图 15-4（b）所示。

在并行转换的实施过程中，首先以现有系统的作业为正式作业，新系统的处理结果作为校核之用，经过一段时间的运行，在验证新系统处理准确可靠后，现有系统退出运行。根据系统的复杂程度和规模大小不同，并行运行的时间一般可在两三个月到一年之间。

采用并行转换的风险较小，在转换期间还可同时比较新旧两个系统的性能，并让系统操作员和其他有关人员得到全面培训。因此，对于一些较大的信息系统，或处理过程复杂、数据重要的系统，并行转换是一种最常用的转换方式。但是，由于在并行运行期间，要两套班子或两种处理方式同时并存，人力和费用消耗较大，转换的周期长，并且难以控制新旧系统中的数据变化。这就要求做好转换计划并加强管理，在新旧系统验证吻合后要及时停止现有系统的运行。

3. 分段转换策略

分段转换策略也称为逐步转换策略，这种转换方式是直接转换方式和并行转换方式的结合，采取分期分批逐步转换，如图 15-4（c）所示。一般比较大的系统采用这种转换方式较为适宜，它能保证平稳运行，费用也不太高；或者现有系统比较稳定，能够适应自身业务发展需要，或新旧系统转换风险很大（例如，在线订票系统、银行的中间业务系统等），也可以采用分段转换策略。

采用分段转换时，各子系统的转换次序及转换的具体步骤，均应根据具体情况灵活考虑。通常可采用如下策略：

（1）按功能分阶段逐步转换。首先确定新系统中的一个主要的业务功能率先投入使用，在该功能运行正常后再逐步增加其他功能。

（2）按部门分阶段逐步转换。先选择系统中的一个合适的部门，在该部门运行新系统，获得成功后再逐步扩大到其他部门。这个首先运行新系统的部门可以是业务量较少的，这样比较安全可靠；也可以是业务最繁忙的，这样见效大，但风险也大。

（3）按机器设备分阶段逐步转换。先从简单的设备开始转换，再推广到整个系统。例如，对于联机系统，可先用单机进行批处理，然后用终端实现联机系统。对于分布式系统，可以先用两台计算机联网，以后再逐步扩大范围，最终实现分布式系统。

分段转换策略的优点是，新旧系统的转换震动比较小，用户容易接受。但由于是采用渐进方式，导致新旧系统的转换周期过长，同时由于需求的变化，给新系统的稳定造成比较大的影响。而且，分段转换策略对系统的设计和实现都有一定的要求，在转换过程中，需要开发新旧系统之间的接口，还需要制订阶段性的转换目标和计划。

15.7.2 数据转换和迁移

数据转换和迁移是新旧系统转换交接的主要工作之一。为使数据能平滑迁移到新系统中，在新系统设计阶段就要尽量保留现有系统中合理的数据结构，这样才能尽可能降低数据迁移的工作量和转换难度。但是，由于新系统的引入，数据迁移工作是个必然的过程，现有系统中的

数据可以通过定制开发的转换工具软件翻译为新系统可以接受的数据格式。

数据转换和迁移工作的原则是数据不丢失。许多无法自动转换的数据，必要时通过手工方式补录进入新系统。数据迁移对系统切换乃至新系统的运行有着十分重要的意义。数据迁移的质量是新系统成功上线的重要前提，同时也是新系统今后稳定运行的有力保障。如果数据迁移失败，新系统将不能正常启用；如果数据迁移的质量较差，没能屏蔽全部的垃圾数据，对新系统将会造成很大的隐患，新系统一旦访问这些垃圾数据，可能会由这些垃圾数据产生新的错误数据，严重时还会导致系统异常。相反，成功的数据迁移可以有效地保障新系统的顺利运行，而且能够继承珍贵的历史数据。

1. 数据迁移的方法

系统转换时的数据迁移不同于从 OLTP 到数据仓库的数据抽取。后者主要将 OLTP 系统在上次抽取后所发生的数据变化同步到数据仓库，这种同步在每个抽取周期都要进行，一般以天为单位。而数据迁移是将需要的历史数据一次或分几次转换到新系统，其最主要的特点是需要短时间内完成大批量数据的抽取、清洗和装载。

数据迁移的主要方法大致有三种，分别是系统切换前通过工具迁移、系统切换前采用手工录入和系统切换后通过新系统生成。

(1) 系统切换前通过工具迁移。在系统切换前，利用 ETL 工具把现有系统中的历史数据抽取、转换，并装载到新系统中去。这种方法是数据迁移最主要，也是最快捷的方法。其实施的前提是，历史数据可用并且能够映射到新系统中。这种迁移方式既可一次实现，也可以分次实现。一次迁移的优点是迁移实施的过程短，相对分次迁移，迁移时涉及的问题少，风险相对较低。其缺点是工作强度比较大，由于实施迁移的人员需要一直监控迁移的过程，如果迁移所需的时间比较长，工作人员会很疲劳。一次迁移的前提是新旧系统数据库差异不大，在允许的宕机时间内可以完成所有数据量的迁移；分次迁移可以将任务分开，有效地解决了数据量大和宕机时间短之间的矛盾。但是分次切换导致数据多次合并，增加了出错的概率，同时为了保持整体数据的一致性，分次迁移时需要对先切换的数据进行同步，增加了迁移的复杂度。

(2) 系统切换前采用手工录入。在系统切换前，组织相关人员把需要的数据手工录入到新系统中。这种方法消耗的人力、物力比较大，同时出错率也比较高。这种方法主要针对新旧系统数据结构存在特定差异的情况，即对于新系统启用时必需的期初数据，无法从现有的历史数据中得到。对于这部分期初数据，就可以在系统切换前通过手工录入。

(3) 系统切换后通过新系统生成。在系统切换后，通过新系统的相关功能，或为此专门开发的配套程序生成所需要的数据。通常根据已经迁移到新系统中的原始数据来生成所需要的结果数据。这种方法可以减少迁移的数据量。

2. 数据迁移前的准备工作

数据迁移的实施可以分为三个阶段，分别是数据迁移前的准备、数据转换与迁移和数据迁移后的校验。由于数据迁移的特点，大量的工作都需要在准备阶段完成，充分而周到的准备工作是完成数据迁移的主要基础。具体而言，要做好以下 7 个方面的工作：

(1) 待迁移数据源的详细说明,包括数据的存放方式、数据量和数据的时间跨度。

(2) 建立新旧系统数据库的数据字典,对现有系统的历史数据进行质量分析,以及新旧系统数据结构的差异分析。

(3) 新旧系统代码数据的差异分析。

(4) 建立新旧系统数据库表的映射关系,以及对无法映射字段的处理方法。

(5) 开发或购买、部署 ETL 工具。

(6) 编写数据转换的测试计划和校验程序。

(7) 制定数据转换的应急措施。

3. 数据转换与迁移

在数据转换与迁移阶段,首先需要制定数据转换的详细实施步骤和流程,准备数据迁移环境。然后要做好业务上的准备,结束未处理完的业务事项,或将其告一段落。接下来使数据转换和迁移涉及的技术都得到充分测试,最后实施数据转换和迁移。

数据转换与迁移程序大致可以分为抽取、转换与装载三个过程。数据抽取、转换是根据新旧系统数据库的映射关系进行的,转换步骤一般还要包含数据清洗的过程。数据清洗主要是针对源数据库中,对出现二义性、重复、不完整、违反业务或逻辑规则等问题的数据进行相应的清洗操作。在清洗之前需要进行数据质量分析,以找出存在问题的数据。数据装载是通过装载工具或自行编写的 SQL 程序将抽取、转换后的结果数据加载到目标数据库中。

数据抽取前,需要做大量的准备工作,具体如下:

(1) 针对目标数据库中的每张数据表,根据映射关系中记录的转换加工描述,建立抽取函数。该映射关系是准备阶段进行数据差异分析的结果。

(2) 根据抽取函数的 SQL 语句进行优化。通常可以采用的优化方式有调整相关参数的设置、启动并行查询、采用优化器、创建临时表、对源数据表做分析和增加索引等。

(3) 建立调度控制表,包括 ETL 函数定义表(记录抽取函数、转换函数、清洗函数和装载函数的名称和参数)、抽取调度表(记录待调度的抽取函数)、装载调度表(记录待调度的装载信息)、抽取日志表(记录各个抽取函数调度的起始时间和结束时间,以及抽取的正确或错误信息)和装载日志表(记录各个装载过程调度的起始时间和结束时间,以及装载过程执行的正确或错误信息)。

(4) 建立调度控制程序,该调度控制程序根据抽取调度表动态调度抽取函数,并将抽取的数据保存到文件中。

数据转换的工作在 ETL 过程中主要体现为对源数据的清洗和代码数据的转换。数据清洗主要用于清洗源数据中的垃圾数据,可以分为抽取前清洗、抽取中清洗和抽取后清洗。ETL 对源数据主要采用抽取前清洗。对代码表的转换可以考虑在抽取前转换和在抽取过程中进行转换。具体转换操作如下:

(1) 针对 ETL 涉及的源数据库中的数据表,根据数据质量分析的结果,建立数据抽取前的清洗函数。该清洗函数可由调度控制程序在数据抽取前进行统一调度,也可分散到各个抽取函数中调度。

(2) 针对 ETL 涉及的源数据库中的数据表，根据代码数据差异分析的结果，对需要转换的代码数据值进行转换。如果数据长度无变化或变化不大，考虑对源数据表中引用的代码在抽取前进行转换。抽取前转换需要建立代码转换函数。代码转换函数由调度控制程序在数据抽取前进行统一调度。

(3) 对新旧代码编码规则差异较大的代码，考虑在抽取过程中进行转换。根据代码数据差异分析的结果，调整所有涉及该代码数据的抽取函数。

4. 数据迁移后的校验

在数据迁移完成后，需要对迁移后的数据进行校验。数据迁移后的校验是对迁移质量的检查，同时数据校验的结果也是判断新系统能否正式启用的重要依据。可以通过以下两种方式对迁移后的数据进行校验：

(1) 对迁移后的数据进行质量分析：可以通过数据质量检查工具，或编写有针对性的检查程序进行。对迁移后数据的校验有别于迁移前历史数据的质量分析，主要是检查指标的不同。迁移后数据校验的指标主要包括完整性检查、一致性检查、总分平衡检查、记录条数检查和特殊样本数据的检查。

(2) 新旧系统查询数据对比检查：通过新旧系统各自的查询工具，对相同指标的数据进行查询，并比较最终的查询结果。先将新系统的数据恢复到现有系统迁移前一天的状态，然后将最后一天发生在现有系统上的业务全部补充到新系统，检查有无异常，并和现有系统比较最终产生的结果。

15.8 现有系统演进

随着信息系统的运行和业务的发展，对现有系统进行演进，如对现有系统进行扩展升级等，使其适应当前的应用情况，就成为必然的，也是经济的选择。而如果企业有多个应用系统，可通过对这些系统进行集成，使数据能在这些系统中共享。有关企业应用集成的知识，在之前章节已有详细讨论，此处不再赘述，本节仅讨论系统的扩展问题，以及扩展与集成的区别。

1. 系统扩展

系统的可扩展性是指将新的功能添加到系统中的能力。可扩展性可以分为动态可扩展性和静态可扩展性。动态可扩展性是指在系统运行的过程中，能够添加新的功能，而不会影响系统的其他部分；静态可扩展性是指在添加新的功能时，系统必须停止运行，当新功能添加之后，系统再重新启动。提高系统可扩展性的方法是在系统架构中减少构件之间的耦合。显然，如果在系统规划和设计时充分考虑了可扩展性的因素，在系统架构上进行了预留，则同类型业务的扩展就相对容易些。

一般地，当客户提出需求变更或增加新的功能时，通常采用的方法首先就是对现有系统进行扩展，以满足这种变化。扩展一般包括延伸型扩展和新建型扩展，前者需要在理解扩展点附近的架构及代码的基础上，以原有方式进行功能扩充；后者则可能会完全另起炉灶，在适应系统整体架构的前提下，增加全新的功能。

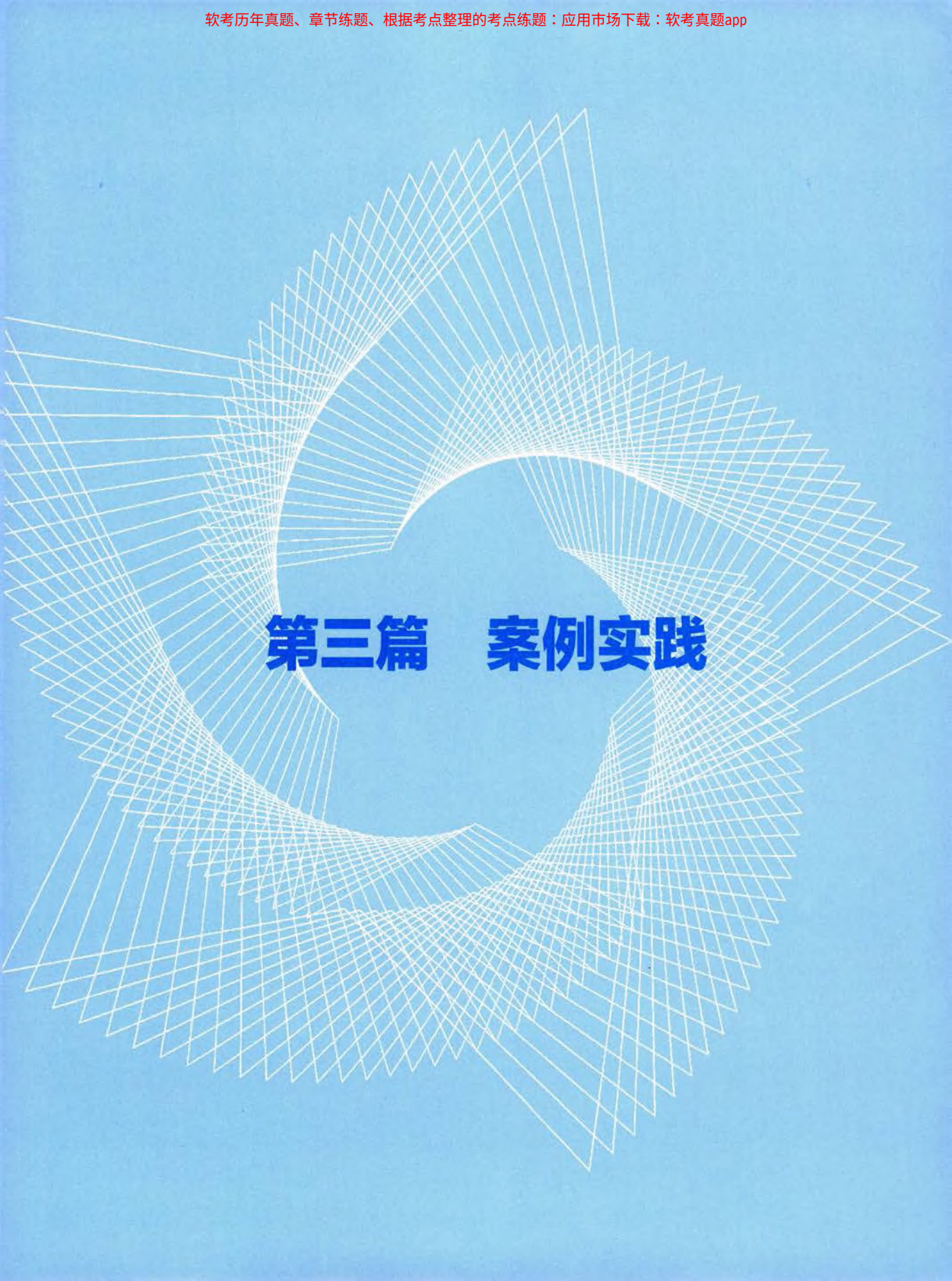
通过在基本软件基础上，引入第三方软件包并进行二次开发的方式，可以迅速对系统功能进行扩展。例如，引入具有手写签批功能的控件，可以立即得到支持领导手工批阅公文的功能扩展。但这种引入也是双刃剑，需要在引入前进行充分的研究和分析，确保其能满足目前的扩展需求外，还具有适度前瞻的特性。同时，要求引入详细的设计文档甚至是源码，以保证对引入包的可控性，避免过度依赖第三方技术支持，降低实施风险。

2. 扩展与集成的比较

系统扩展和集成分别属于深度维护和广度维护活动，和开发过程类似，通常也需要经过需求分析、设计、编码、测试和实施的各环节来共同保证系统扩展和集成活动的最终成效。因此，需要分析人员、设计人员、编码人员、测试人员和实施人员参与其中，同时，也需要过程管理人员进行项目管理。此外，系统集成还特别需要组织的高层人员参与协调与沟通。

系统扩展的重点在设计阶段，为达到平滑扩展，需要仔细研究扩展点附近的软件环境，要避免因扩展引起原有系统的不稳定或功能失效，要进行细致的回归测试；系统集成的重点在分析阶段，为达到无缝集成，需要仔细分析业务，尤其是业务关联点，避免过度耦合、深度介入，增加集成复杂度。另外，还需要组织的高层领导强势协调，强调全局观念，互相配合，方能顺利进行集成。

在系统测试方面，系统扩展和集成的测试要进行全面的回归，不能“改头测头，改脚测脚”，系统集成尤其要重视接口的测试和全流程的测试。



第三篇 案例实践

第 16 章 Web应用系统分析与设计

Web 应用系统是指用户界面驻留在 Web 浏览器中的任意应用程序，它基于 Web 技术和 W3C 标准，通过一个用户界面（Web 浏览器或支持 Web 技术进行访问的可视化界面）提供 Web 特定的资源，如内容和服务等。随着新的 Web 平台、技术和标准不断演进，Web 应用的范围和复杂性千差万别，从小规模应用到大规模分布在 Internet、Intranet 和 Extranet 的企业应用，现在的 Web 应用系统提供了巨大的、各种各样的功能，并且有不同的特性和需求。Web 应用系统能使客户和商业公司具备在线商务处理、供应链集成、动态满足客户要求以及个性化服务等能力，系统的架构要求是健壮的、可伸缩的和可扩展的，可以提升性能并且能处理大量不可预知的商务交易负载。

本章在介绍 Web 应用系统的基础上，从 Web 应用架构设计、Web 应用开发技术框架、Web 应用系统开发过程及测试等方面全面介绍 Web 应用系统开发的相关知识。

16.1 Web 应用系统简介

16.1.1 Web 应用程序

Web 应用程序是在 Web 浏览器中运行的软件。企业在远程交换信息和提供服务时使用 Web 应用程序，以方便、安全地与客户联系。购物车、产品搜索和筛选、即时消息和社交媒体新闻源等最常见的网站功能均设计为 Web 应用程序，这些应用程序允许用户在不安装或配置软件的情况下访问复杂的功能。

Web 应用程序拥有多个优势，几乎所有大企业都将其作为向用户提供的产品和服务的一部分。以下是 Web 应用程序的一些最常见的优势。

(1) 易于访问。可以从所有 Web 浏览器及各种个人和企业设备访问 Web 应用程序。位于不同地点的团队可以通过基于订阅的 Web 应用程序访问共享文档、内容管理系统和其他业务服务。

(2) 开发高效。具体而言，Web 应用程序的开发过程相对简单，且对企业而言具有成本效益。小型团队可以实现较短的开发周期，使 Web 应用程序成为一种构建计算机程序的高效且经济的方法。此外，由于同一版本适用于所有现代浏览器和设备，因此开发人员无须为多个平台创建多个不同的迭代。

(3) 使用简单。Web 应用程序无须用户下载，易于访问，同时使得最终用户无须进行维护和担心硬盘驱动器的容量问题。Web 应用程序可自动进行软件和安全更新，这意味着它们始终是最新版本，并且面临的安全漏洞风险更低。

(4) 扩展性强。使用 Web 应用程序的企业可以在需要时添加用户，而无须配备额外的基础设施或昂贵的硬件。此外，绝大多数 Web 应用程序数据都存储在云中，这意味着企业无须投资